

AI Agent Memory Infrastructure: A System Design Manuscript

The RAM Side and Its Periphery — Interview-Depth Edition (derivations, invariants, spoken skeleton, mechanism pre-study, follow-up depot)

2026-07-11

0. Problem Setup and Global Anchoring

Target role: ByteDance AI Agent Memory Infrastructure, Senior SWE. JD core responsibilities: full memory lifecycle — ingestion, storage, indexing, retrieval, updating, compression, forgetting; hard-problem list: memory representation, retrieval & ranking, conflict resolution, summarization & fusion, lifecycle management, multimodal fusion.

Master anchor (the single anchor of this manuscript; every number derives from it):

Quantity	Value	Derivation
User scale	1B DAU	Given
Message volume	240 msgs/user/day	Given
Message size	2KB raw	Back-derived from 175PB/yr
Daily total	480 TB/day raw	$1B \times 240 \times 2KB$
Yearly total	175 PB/yr raw	$\times 365$
Message QPS	2.8M/s mean, ~8M/s peak	$1B \times 240 / 86400$, peak factor $\sim 3\times$
Concurrent sessions	~167M mean	Little's Law: $1B \times 4h / 24h$ (4h/day online = $240 \text{ msgs} \div 1 \text{ msg/min}$)

Anchoring discipline (validated repeatedly through this exercise): every new estimate must collide with the master anchor before acceptance; changing the anchor is a global event — all downstream numbers scale together; old and new anchors must never be mixed. Two bookkeeping slips in this exercise (a "2M concurrency" side-estimate; a 0.5KB/msg drift) were both caught by this discipline.

1. Storage Tiering Panorama (conclusions from the prior session)

1.1 Three-tier ledger

Tier	Contents	Window	Compression	On-disk	Access pattern
Hot RAM	Active session state	Active period + TTL tail	2-3:1 (record-level)	Steady 20-50TB	Point/range reads; read-bandwidth bound
Warm SSD	Last ~3 months raw text	~90 days	5:1 (ORC batch)	~8.8 PB	Cache-miss reload, recent lookups, incremental refinement input
Cold HDD	Full history	1yr+/retention	5:1 + EC 1.4 $\times$	~49 PB/yr	Pure batch: replay, re-refinement, compliance

Key rulings:

- **Compression ratio is a tier property, not a global constant:** large-block batch compression buys high ratio (5:1–8:1 in the warehouse); record-level small-block compression buys random access (2-3:1 hot). Conversation corpora (templated LLM answers, cross-session repetition) with a custom-trained dictionary can reach 8:1; re-anchoring shrinks warm to 5.5PB and cold to ~31PB globally.

- **Replication factor per tier:** $3\times$ replication only for hot/warm serving tiers (read throughput + fast recovery); cold uses erasure coding (RS $10+4$, $\sim 1.4\times$), halving cost. Cold is cheap and not the crux — one sentence when asked about cost, never proactively expanded.
- **The RAM tier never bulk-loads:** serving retrieves top-k (KB-MB working sets + hot indexes in memory); 8.8PB cannot fit in RAM and does not need to.

1.2 Dual path + backfill (the architectural spine)

A conversation does not "live in some tier"; the same bytes travel two paths feeding two systems:

1. **Online path (TAO-style, low latency):** one of the double-writes $\rightarrow$ online KV/graph (ms-level), serving display + recent context at inference;
2. **Offline path (DataSwarm/Hive-style):** the same message log lands in the warehouse (full ORC); Presto/Spark mining refines embeddings/summaries/profiles;
3. **Backfill loop:** refined memory pushed back to the online serving tier (vector index/KV) for low-latency top-k retrieval at inference.

Speak the spine proactively in interviews; the internals of each path (TAO shard affinity, ORC encoding, Presto MPP, backfill pipelines) stay in the pocket until asked. **This session drew a boundary: TAO's UI-display duty is out of scope; only "online KV holds the recent window" remains — inference-related concerns are this team's border.**

2. What Inference Actually Stores: The Recompute Trichotomy

The first main line of this session. One invariant as the criterion:

Recomputable and cheap $\rightarrow$ don't store; recomputable but expensive with high reuse / structural dependence on precomputation $\rightarrow$ store (as cache/index); non-recomputable $\rightarrow$ must store (as log).

2.1 Do not store QKV/activations (two hard reasons)

1. **Zero marginal information (the essential reason):** the last layer's hidden state is a deterministic function $f(\text{raw text, model weights})$ — precisely recomputable given text and weights, hence zero marginal information relative to raw text. It is not data; it is cached computation. The value of training data lies in containing what the model does not have (what the user said, which answer was accepted) — all of which lives in raw text. Nor can stored activations be used for training without recomputation: backprop needs all lower-layer activations rebuilt in the forward pass; a stored last layer cannot be inserted into that flow.
2. **The magnitudes fail:** KV cache $\approx 320\text{KB/token}$ at 70B scale ($2 \times 80 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ dims} \times 2\text{B}$) versus raw text $\sim 4\text{B/token}$ — a **10^5 gap**. Plugging the anchor: $1.2 \times 10^{13} \text{ tokens/day} \times 320\text{KB} \approx 4 \text{ EB/day}$; even last-layer-only is $\sim 380 \text{ PB/day} \approx$ two years of compressed raw text per day. And KV is model-bound: one model upgrade voids the entire inventory.

The legitimate outlets for the "last layer has information" intuition: distillation stores top-k logits ($\sim 100\text{B/token}$; genuinely informative to a student lacking teacher weights; stored for sampled subsets only); semantics are already captured as pooled embeddings ($1\text{--}4\text{KB/chunk}$).

2.2 The converged inventory

1. **Raw text:** source of truth, model-agnostic; every model generation re-refines from it — category three (non-recomputable world input);
2. **Inference-time context snapshot log:** the full contents that actually entered the context (retrieval results, system prompt, tool outputs, profile snapshot) + model version + sampling params. **A function of time-varying system state — unreplayable after the fact.** This is the core training data for the memory retrieval/ranking system, the exact analogue of ads feature logging guarding against training-serving inconsistency;
3. **Embeddings (assigned to the retrieval block):** recomputable but cheap ($\sim 1\text{PB}$), globally high-reuse, and retrieval structurally requires the index to exist before the query — identity: **materialized index**, kin to the inverted index (rebuildable from text, yet nobody skips storing it). Tagged with embedding-model version; upgrades require full reindex;
4. **Not stored:** QKV (the anti-example of category one: recomputable + $10^5\times$ more expensive + reuse confined to one session); the model itself (a registry pointer).

KV cache's correct station: the inference engine's tiered serving cache (HBM→CPU RAM→NVMe), minute-to-hour TTLs, session/prefix granularity, never entering the warehouse.

Note the architectural split in embedding production: pooling a decoder LLM's hidden states yields poor retrieval embeddings (causal-LM objective ≠ similarity objective; anisotropy). Production uses an independent contrastively-trained embedding model (two-tower/EBR). Two models, two paths; the interface is text; each evolves independently.

3. The Retrieval Map: Two Halves and the Signal Production Line

3.1 Partitioning the map

- **Internal memory (owned corpus):** full-stack control — build indexes, hybrid retrieval, train two-towers, reindex;
- **External search (black-box API):** no indexing rights; control points shrink to the two ends — inbound query rewriting (conversation context → search-friendly query) and outbound rerank (encode the returned top-50 on the fly; ANN's structural constraint kills retrieval, not reranking).

What unifies: the **interface layer** (both become retrieve(query) → passages + source + score tools); the **fusion layer** (a cross-encoder as the common confluence, candidates from both halves competing on one scale); the **signal** (utilization logs are half-agnostic; one log feeds query gen, two-tower, router, and reranker). What never unifies: index ownership (a physical constraint).

Query side: "understand once, render twice" — shared intent extraction (coreference, entity completion), forked rendering: the internal head preserves the user's private vocabulary (the alias table personalizes this branch); the external head normalizes to web terminology + full entity names + temporal qualifiers. **Unify at semantics, fork at vocabulary — and only one vocabulary is private.**

3.2 The v0 ruling: lexical + LLM query rewriting

- **Scenario-specific reason:** in personal-memory retrieval, query and corpus come from the same person's word distribution — users search for their own words; vocabulary mismatch is inherently weak; lexical is inherently strong;
- **Benchmark evidence:** BEIR's core finding — zero-shot BM25 crushes a large cohort of dense models out of domain; dense wins reliably only with in-domain training data;
- **ROI:** zero training dependency, one week to ship, no pseudo-query noise.

Dense's entry condition: relevance signal at scale. This scenario lacks native clicks — three congenital defects:

1. **No SERP:** users never see "which memories were recalled"; no click in the behavioral sense exists; utilization is a model-side inferred label;
2. **The corpus is per-user:** signal cannot pool at the doc level (doc-ID embeddings, popularity priors, query-doc co-occurrence all vanish); learning must live in attribute/content space and transfer across users — the same predicament and remedy as Google's personal-search literature (Gmail/Drive);
3. **Sheer volume:** one inferred label per turn.

Hence **if a two-tower is ever trained, the doc tower carries no memory-ID parameters — pure content encoding + attribute features**. The forced constraint aids transfer, and retroactively strengthens v0: even the model is pushed into content space, where lexical already lives.

3.3 The signal production line (five tiers + the session layer)

A cascade from cheap-and-weak to expensive-and-strong — never "pick one," always cascade — the recommender playbook (impression→click→conversion→human eval) reskinned:

Tier	Signal	Source & cost
0	Self-supervised pseudo-task (conversational ICT: turn N as answer, back-derive query)	Free; cold start; known distribution gap
1	Execution feedback (empty recall / score distribution / agent retry = free negative labeler)	System-generated, available at launch, largest volume — the staple
2	Utilization (did the answer depend on the chunk; LLM judge / citation markers)	Sampled precision labels — the finetune diet

Tier	Signal	Source & cost
3	User outcome signals ("didn't I tell you X" = strong negative; regenerate / smooth progression)	Free but sparse, delayed, noisy; attribution hard (credit assignment)
3.5	Session implicit feedback: the next prompt scores the previous query — reformulation = negative, deepening follow-up = positive, topic switch = unlabeled. Dense (every turn), but must be disambiguated by lower tiers (bad query / absent knowledge / generation misuse — three culprits). Use as reward, not label (the next prompt is contaminated by the observed failure — distribution shift)	Free; one order denser; a direct transplant of twenty years of web-search query-reformulation-as-implicit-feedback
4	Gold set (human labels + counterfactual ablation)	Most expensive; used only for evaluation and calibrating the four tiers above

The accompanying pathology transfers wholesale: only recalled items are observed → bandit feedback/IPS; the LLM favors top-ranked chunks → position bias/PAL; adoption signals are delayed and noisy → delayed feedback.

Query-generation training ladder: v0 prompt (history in context = the personalization prior taken for free from unsupervised LM) → v1 rejection sampling (keep only retrieval successes) → v2 RL with verifiable retrieval reward (the Search-R1/DeepRetrieval line — the reward comes from downstream outcomes; the query distribution is the policy's own exploration, sidestepping the pseudo-query gap). **Representation comes free from unsupervised learning; policy must be paid for — the currency is the utilization log.** LM objective ≠ query utility objective (search queries never appear in natural dialogue; good queries need discriminative terms, and high-probability terms are precisely the non-discriminative ones — the reason TF-IDF exists).

The agentic-search vs. embedding-retrieval boundary: agentic spends tokens to save an index (Claude Code: pure iterative grep); embedding spends an index to save tokens (Cursor). Low-QPS, high-value, iterable → agentic wins; at 1B users with retrieval on every turn's critical path, query gen is the retrieval layer's front end, not its replacement. The ledger is QPS, not quality.

The incumbent advantage, stated precisely: Gmail search has real SERP clicks — twenty years of genuine per-user-corpus click data; models (attribute-level relevance, query understanding) warm-start into agent memory. But compliance walls (purpose limitation) mean the asset lives in models and methodology, not raw content. For players without that inheritance (ByteDance), signal building is not optional — it is the construction site of the moat, and the very reason this team exists.

Interview capstone paragraph: retrieval v0 is lexical + LLM query rewriting — same-person vocabulary, zero training dependency, best ROI. I know how to train dense (two-tower / in-batch negatives + logQ / hard-negative curriculum / IPS debiasing / Margin-MSE distillation), but its precondition is relevance signal at scale, and this scenario has no native click — no SERP, per-user corpus, inferred labels — so dense is not a day-1 component; it is the v1 after signal building succeeds. How to manufacture that signal is itself the hardest sub-problem of this system.

4. Industry Alignment (mem0 as the baseline; named in the JD)

The six-step skeleton holds entirely; three deviations corrected (all on the write side):

Matches: read path = recent messages + summary + retrieved top-k concatenated into context → generation; retrieval is hybrid (semantic vectors + BM25 with lemmatization + entity matching + temporal-intent scoring, rank-fused into top-K) — lexical is present, the v0 ruling verified verbatim; write-back is async and never blocks generation.

Three deviations:

- Refinement triggers per-turn incrementally, not at session end:** mem0 fires extraction on each message pair (context = conversation summary + recent window); the summary is refreshed by an independent async module. Correct form: dual-track — per-turn async fact extraction (primary) + periodic summary (auxiliary);
 - Conflict resolution drifted from write-time to read-time:** the paper version judges ADD/UPDATE/DELETE/NOOP at write; the production version switched to **ADD-only** — old and new facts coexist with temporal metadata + contradiction marks; write-time does hash-dedup only; contradictions are settled at retrieval via temporal reasoning. Blunt beats clever;
 - Task-clustered context selection is beyond standard:** industry default = recent window + summary + retrieved supplement; present per-task threading as a highlight, not the default architecture.
-

5. E2E Workflow: T0–T6 Overview

A prompt's full journey — six hops mapped to JD duties:

Hop	Action	JD mapping
T0	Read session state (recent turns + summary), local memory	storage / indexing (consumer side)
T1	Routing + query generation (point-read profile/alias)	retrieval front end
T2	Retrieval: local inverted index + vector dot-product / external API → rerank confluence	retrieval & ranking
T3	Context assembly: four streams within a token budget	context engineering (named in JD)
T4	Call inference (RPC, another team); context over the wire	— interface
T5	Async write-back, four items: append state, dual-write warehouse, context snapshot log, fact extraction	ingestion + storage
T6	Periodic batch: refinement + compaction	updating / summarization / conflict / compression / forgetting / lifecycle

Reads only on the critical path (all writes deferred to async T5/T6) — the first design principle of the whole workflow. T5/T6 is this team's heart (what distinguishes it from a pure storage team); half of T0–T4 is commodity.

Peripheral interface inventory (the outside world as seen by the core): ingest(conversation_delta) (at-least-once, bounded disorder) + T+1 warehouse replayability; retrieve(user_id, query_context, budget) → ranked memories (with provenance) + get_profile; rented services = storage engines + GPU inference (RPC + quota) + external search/routing/generation quality (neighbor teams); cross-cutting = context snapshot log + forget(user_id, scope), the GDPR cascade port constraining all storage design from day one.

6. T0 Deep Dive: Session State (the heaviest hop of this exercise)

6.1 Store selection and schema

- **Ordered KV (wide-column), LSM engine:** ByteDance idiom Abase/Bytable; open-source analogue ScyllaDB/Cassandra (Discord's trillion-message store is isomorphic); Meta lineage MyRocks/ZippyDB. LSM dividends: append-friendly writes; newest data naturally in memtable/upper levels (the "recent conversation" read pattern eats this for free); native TTL (expired data dropped at compaction).
- **Key = (userid, sessionid[, time_bucket]),** time-descending; clustering-key ordering gives range scans for free. **taskid stays out of the key** (at query time taskid is not yet computed — classification depends on the context being read; keying on it is a circular dependency). Correct read: **bounded read** (last N / last T minutes / memory cap, whichever first) + app-layer task filtering (N is in the hundreds; app-side filtering is free). time_bucket follows Discord: prevents ultra-long sessions from blowing a partition.
- **Durability grade:** write to memory replicas (3-replica ack) + async append-log to disk — durability by replica redundancy rather than synchronous flush; loss window in milliseconds, at most the final message (user resends). Session state is not ledger data.
- **Task-clustering system contract** (the model belongs to the algorithm team; this team sets the constraints): query embedding dotted against K in-session centroids (K single-digit, microseconds); low-confidence short prompts ("what about the second paragraph?") → **sticky to thread** (inherit last turn's taskid — elliptical prompts occur precisely when the topic has not switched; the conversational-continuity prior comes free); new cluster opens in O(1) below a similarity threshold.

6.2 Capacity: Little's Law, use one of three (byte stock)

Single session state $\approx 240 \times 2\text{KB} = 480\text{KB}$ raw → $\sim 110\text{KB}$ compressed (with centroids + metadata).

The core clarification (a genuine stuck point in this exercise): 480TB/day is **flow**, not **stock**. The hot pool is a pass-through buffer, not a distillery — no filtering, no extraction, residency only; each byte soaks only for its session's lifetime ($\sim 4\text{-}5\text{h}$ incl. TTL tail):

Stock = inflow rate $\times$ residency = $(480\text{TB}/86400\text{s}) \times \sim 16000\text{s} \approx 90\text{TB}$ raw → 2-3:1 → **30-45TB**

Locking with the concurrency reckoning ($167M \times 110KB \approx 18TB$). "All of a day's data simultaneously in RAM" never happens; 160TB (the all-day random-access version) is a counterfactual upper bound whose $2.7\times$ premium over the 60TB batch version is precisely the price tag of random access.

6.3 Eviction: the victory of blunt TTL/LRU (post-alignment version)

Industry evidence: Discord/Messenger keep no session-aware RAM pool — durable LSM as primary store (newest data in memtable; SSD point reads at ms are enough for serving), a generic TTL cache in front, eviction left to LRU; the AWS reference = Redis(TTL) + DynamoDB + batch summarization. **Nobody writes the "evict only after refinement completes" clause** — refinement consumes the warehouse/log path, never the RAM copy; the two paths were always decoupled.

Work solution (capstone):

50TB quota + 2h sliding TTL (from last activity — chat workloads reset expiry on every write, so fixed and sliding coincide) + jitter + volatile-lru + durable-tier miss reload as the safety net.

Four parts, four duties: TTL for routine eviction, LRU for allocation under pressure, quota as the guardrail, reload for correctness. The 2h value sits squarely in the industry mainstream (30min–2h). TTL selection is at heart a return-gap-distribution problem (heavy-tailed, knee of the CDF); industry's lazy solution is superior: set TTL as an upper bound and let volatile-lru adapt online — **LRU is itself an online nonparametric estimator of return probability.**

6.4 Miss rate: the denominator check and the burstiness-memory conjugacy

Denominator discipline: hit rate is per access, not per segment (an external AI's table counted only segment-head loads, discarding hundreds of in-segment hits — conclusions off by an order of magnitude). Re-cut: all scenarios 98%+; the "~0%" row is physically impossible under the anchor (240 messages pairwise 2h apart needs 480 hours).

Structure: under sliding TTL, in-segment gaps (1 min « 2h) never expire; misses occur only at **segment heads with gaps > 2h**, and the count of long gaps is locked by circadian rhythm (overnight + 2-3 daytime troughs $\approx 3\text{--}4/\text{day}$):

miss rate $\approx 3.5/240 \approx 1.5\%$ (range 0.4%–2.5%; extreme-fragmentation ceiling 8%)

Miss cost: 40K miss/s $\times$ 100KB reload $\approx 4GB/s$ onto the durable tier — noise; single penalty a few ms at segment head, invisible to users.

The uniformization thought experiment closes the loop: were the 240 messages uniform (one per 6 minutes « 2h), sliding TTL never expires and misses vanish — but the zero-miss world is unaffordable: universal residency = $1B \times 110KB = 110TB$, blowing the quota.

Miss and memory are conjugate: uniform = zero miss at full residency; bursty = 3-4 misses buying a 5 $\times$ memory saving — the user's sleep is our memory budget; segment-head misses are the receipts. TTL is where the knife falls between the conjugate pair. The capacity argument thus closes threefold: the mean ledger (Little's Law, $20TB < 50TB$), the distribution ledger (conjugacy forbids sliding into full residency), the ordering ledger (80/20 heavy tail: residency is per-head, hits are per-traffic; LRU naturally spends memory on heavy users — flow-weighted hit rate > 99%).

Exact vs. engineering solutions: given 30-day per-user activity, the linear-expectation formula is an undeclared approximation — residency is a **functional of the gap distribution** (the clump/busy-period process under sliding TTL: $P(\text{gap} > \tau)$ sets clump count, the truncated mean sets clump length; the same 4h/day spread uniformly vs. in three blocks differs $4\times$ in residency), analytically hard under heavy tails. Engineering skips analysis: **trace-driven simulation** — replay 30 days of real access logs through the TTL+LRU logic; steady state, miss, quantiles, and parameter sweeps (the miss-memory curve over τ , the CDF knee drawn rather than solved) all read off directly; hang it as a periodic regression job so behavioral drift alarms before production does. **Analysis proves structure; simulation produces numbers.**

6.5 Architecture: compute-follows-data (a) vs. stateless serving (c)

Physical criterion: are the CPU reading the data and the RAM holding it in the same machine? Same machine = memory bus ($\sim 100GB/s$, 100ns), NIC irrelevant; cross-machine = both NICs traversed ($25GbE \approx 3GB/s$, 100 μs), $\sim 30\times$ slower.

Two orthogonal decisions: routing requests by userid (one line of config; everyone should) $\neq$ co-locating serving processes with KV shards (the actual switch of the network tax).

Read-bandwidth ledger ("read-heavy means bytes"): reads ~100KB context vs. writes ~2KB — $50\times$; $2.8\text{M reads/s} \times 100\text{KB} \approx 300\text{GB/s}$ **aggregate**, 3.5GB/s per KV node at 80 nodes — 25GbE saturated. Remedies by ROI: (1) delta read (serving-side session-level soft cache + cursor-based incremental reads; in-segment 2-4KB; $300 \rightarrow 12\text{GB/s}$, $25\times$) — a cache, not a replica (T3 assembly holds this memory anyway; "just don't discard it"); loss degrades to a full read, correctness intact; (2) ship compressed blocks; (3) scale out (buying memory machines for their NICs — crude).

Scheme (a), compute-follows-data: a front routing layer fans out once by userid $\rightarrow$ assembly on the data-holding blade; reads ride the memory bus; the read tax vanishes. **At the 4000-blade scale it strengthens**: $2\text{PB} \div 500\text{GB usable} = 4000$ blades; 50TB session state parasitizes 13GB/blade (2.6%); a 1TB machine splits 250GB shard + 500GB compute + 250GB system; the per-user family (state, inverted index, vectors, facts) co-locates by userid — T0-T3 fully local; hotspot drainage via fine-grained virtual shards. Costs on the ledger: rolling restarts now touch stateful nodes (shard responsibility must move first); serving elasticity is chained to sharding (re-sharding cadences locked — long-stable KV shards vs. minute-level serving elasticity).

Scheme (c), the industry default: stateless serving + remote KV + delta read; a 12GB/s tax buys independent scaling of the two layers ("stateless compute, state sinks down").

Ruling: both are legitimate hills; the choice is organizational (storage and orchestration in one team $\rightarrow$ (a)'s coupling tolerable; across teams $\rightarrow$ (c)'s interface decoupling pays). Default to (c) in interviews; keep (a) pocketed for extreme latency budgets. **Routing affinity: yes. Deployment affinity: depends. CPU compute follows data; GPU compute follows the batch** (GPU efficiency lives on cross-user batching — inherently centralized, never moved).

The context $\rightarrow$ GPU leg is unavoidable for everyone: $2.8\text{M} \times 300\text{KB}$ (fully assembled context) $\approx 840\text{GB/s}$ — workload-intrinsic flow; architecture only chooses where extra tax is paid. Inference-side: a million-GPU-scale fleet ($2.8\text{M QPS} \times \sim 1\text{ GPU-sec/turn}$); 840GB/s over tens of thousands of inference nodes = tens of MB/s each — **the network is not the hard part; the million GPUs are**. The real cleaver: **prefix caching** — session-affine routing + increment-identifiability (seqno); the shared prefix's KV already resides in HBM; each turn ships ~10KB of increment; $840 \rightarrow \sim 30\text{GB/s}$. **Delta read's mirror image at the inference layer**; the cross-team interface in one sentence: "we guarantee session-affine routing and identifiable increments; inference harvests the prefix-cache dividend."

7. T1-T4: The Read Path

7.1 T1: decisions eliminated or amortized (post-alignment)

A standalone λ controller + value estimator = over-engineering relative to industry. Two layers instead:

- **Hot layer (profile + high-confidence facts + recent turns): injected unconditionally every turn, zero decision** — a few hundred tokens; judging costs $\geq$ injecting; the decision is **eliminated** (ChatGPT memory's exact form);
- **Cold layer (history retrieval / external search): exposed as tools; the generation LLM decides** — the judgment rides the generation call that happens anyway; marginal cost zero; the decision is **amortized**. "2.8M/s of judgments dearer than generation" is a pseudo-problem: no standalone judgment call exists.

Pre-router's true identity: a 10ms model/rule is not a decider but a **prefetch trigger** — at high confidence, fire retrieval in parallel early, hiding 3-5s external searches inside generation latency; authority stays with the tool call; wrong guesses are recoverable (a wasted search / an agent retry). Bing Chat's search-intent classifier is the precedent. **The router manages speed; the tool call manages correctness**. Academic gating (FLARE/Self-RAG/prefix-logit entropy) = pocket knowledge, not default architecture. The λ framework survives as one bone: tiering by cost (free = always inject; expensive = on demand); budget control degenerates to coarse quotas.

7.2 The fact layer

- **Form**: structured plaintext entries ("user lives in Jersey City") + temporal metadata + an embedding index (plaintext is the substance; the embedding is only its index). Origin = T5 per-turn async LLM extraction + explicit "remember X."
- **Size**: ~50-200B text + 1KB embedding (1024-dim int8; 5000 dims is out of bounds — production mainstream is 384-1536; a per-user small corpus cannot support it, samples < dimensions); design envelope 300KB/user/30d $\rightarrow$ **300TB total**, hot portion (loaded with active sessions) **~50-100TB**, version buffer (multi-embedding coexistence: upgrade dual-version $2\times$, multi-task spaces, A/B) $\rightarrow$ quota **200-300TB**.
- **Forgetting $\neq$ TTL (this team's ontology)**: "allergic to peanuts" must not expire after a decade of silence. **TTL governs caches; supersession + decay govern memory**: superseded (new facts mark old — ADD-only + temporal adjudication), decayed

(down-weighted, never deleted), quota-evicted (per-user hundreds-to-thousands cap; hitting it triggers **compression** — shard-facts merged into a summary — not deletion; the quota signals "time to refine," it is not a hard gate).

7.3 T2: retrieval execution

- **A per-user corpus needs no ANN:** 240/day $\times$ a year $\approx 10^4$ – 10^5 vectors; brute-force dot product is milliseconds on CPU — exact inner product is the terminal solution; HNSW belongs to global billion-scale corpora (user prompts searching shared libraries — videos/posts/friends — revive it; that is recommendation retrieval, another team).
- Order: current session first $\rightarrow$ cross-session KNN serially + time decay; internal per-user inverted index (offline batch build + near-realtime append + periodic compaction) + vector hybrid; external = API.
- Rerank criterion: does this knowledge carry a clue for this question — the judge is the downstream LLM (utilization); v0 judges with the LLM, v1 distills to a small model.

7.4 T3/T4: assembly and transport

- **In-flight ledger (Little's Law, use three of three):** 2.8M/s $\times$ response time. 60s cap $\rightarrow$ 168M in flight $\times$ 300KB = 50TB cap; ~10s mean $\rightarrow$ **~8TB typical**. Report: budget by cap, expectation by mean.
- **Two SLO grades** (industry timeout alignment: search APIs P50 0.45-2s; aggressive tool timeouts 3-5s; interactive TTFT < 500ms; single turns 220ms–7.5s; multi-tool 10-18s): **interactive (default)** = search timeout 3-5s, ≤ 1 external round, whole turn < 10s; **deep (agentic)** = 60s / 10s per search / ≤ 3 rounds. Grade selection is another thing the tool call decides in passing.
- **VRAM enters this team's design as the token budget:** context length = the counterpart's KV-cache VRAM burn (concurrency $\times$ length $\times$ per-token KV sets the GPU count); T3's budget is back-derived from inference-side VRAM/cost — half of compression/refinement's reason to exist ("convey the most memory in the fewest tokens").
- T3 output = system prompt + recent turns + retrieval results + profile digest; budget slicing + provenance ordering; consumption reported upstream (tokens burned, tokens left) — discretion stays there: **we are the meter, not the tollbooth**.

8. T5/T6: The Write Side (this team's heart)

8.1 T5: four async per-turn writes

After generation, asynchronously (zero writes on the critical path); ten-minute latency is tolerable — freshness holds because **what was "just said" is covered by recent turns** (already in context); a ten-minute-late fact library affects only cross-session retrieval, and the next session is tens of minutes away — the async window nests in a natural gap.

1. Append this turn's (prompt, answer) to session state (2KB/write; mature KV shrugs);
2. Dual-write to the warehouse (raw-text ground truth);
3. **Context snapshot log** (append-type; possibly embedding features);
4. Fact extraction (remote async call; see 8.3 for the cost structure).

Initiated on the session blade, direct RPC (skipping the API gateway hop — one queue + serialization saved); **GPU compute never moves here** — batching demands centralization: "CPU compute follows data; GPU compute follows the batch."

8.2 T6: two legs (T5 keeps it fast; T6 keeps it from rotting)

Structural analogy: T5/T6 = LSM memtable append / compaction. Append escapes write latency (ADD-only, message-pair myopia), borrowing consistency and thrift as debt; T6 repays. The default of not repaying: six months in, thousands of facts per user, ~60% near-duplicate or stale; top-k crowded out by junk; **precision collapses silently** (no errors — just dumber answers).

Leg one (refinement outputs): atomic facts aggregated into user profiles/habits/alias tables (cross-turn, cross-session patterns invisible to single-turn myopia — profiles are NOT completable in T5; T5 extracts atoms, T6 refines molecules), embedding recomputation, the session $\rightarrow$ daily summary pyramid (the summary is itself an input to T5 extraction — a self-consuming loop demanding an independent cycle). = the recommender's user-side feature pipeline, with click streams swapped for dialogue streams.

Leg two (compaction, the library's GC) — the JD's compression / forgetting / conflict resolution / lifecycle are these four items by name:

1. **Semantic dedup** (= entity resolution): new fact's embedding dotted against the user's library (sub-ms; the async path makes tens of ms lavish); above threshold $\rightarrow$ merge/cluster;

2. **Contradiction adjudication** (dedup's evil twin: semantically related, mutually exclusive — "works at Meta" vs. "laid off from Meta"; high similarity $\neq$ contradiction — "likes coffee / likes dark roast" is refinement; NLI-grade judgment — the model is the neighbor team's, the pipeline is ours): write-time ADD-only + temporal metadata + contradiction marks; read-time temporal reasoning (newest wins, old retained as valid history); T6 materialization: hot contradiction pairs batch-pre-adjudicated = a materialized view of read-time adjudication;
3. **Decay re-scoring + quota compression**: score = importance $\times$ confidence $\times$ time decay — governs retrieval ordering and compression priority, never deletion; on quota, low-score shard-facts merge into a summary (precision down, never zeroed);
4. **Replay channel** (backfill): after extractor upgrades, re-refine from raw text — the moment the raw-text ground truth pays for itself.

Shared skeleton: **three cases of new-old knowledge relations — duplicate (merge), exclusive (mark + read-time adjudication), stale (down-weight + compress)** — one T6 batch pipeline runs them in order.

Boundary (mechanism vs. policy): this team owns mechanisms (pipelines, schemas, scheduling, replay — the how); the algorithm team owns policy parameters (what counts as contradiction, decay coefficients, importance definitions — the what). Analogy: recommender infra builds the feature store and sample streams, never the ranking objective. **Carving leg two out as "business logic" gets pinned by JD verbatim — without leg two, this team degenerates into a pure storage team, and pure storage is the neighbor's turf.**

8.3 Cost structure: a three-tier funnel, GPU shrunk to the apex

Industry evidence: extraction-class tasks are retreating from small LLMs back to CPU encoders — **the GLiNER family** (<500M encoders, zero-shot NER/IE matching LLMs, efficient on standard CPUs, deployed at scale for PII); distillation (LLM teachers label, BERT students serve — the bootstrap loop's own history is the distillation set; the pipeline feeds itself); NLI has a CPU grade (TinyBERT cross-encoders, ~50ms on CPU, within 3% of GPU). CPU's border: 7B+ generative inference does not hold.

1. **CPU encoders (80-90% of the work)**: embeddings, dedup dot products, GLiNER-style explicit fact extraction, NLI candidate screening (embedding similarity + entity match pre-filter) — **on our own 4000 blades' idle trough cores, data local, zero network**;
2. **GPU small models (the waist)**: screened hard cases — implicit-fact inference, multi-turn summary synthesis;
3. **GPU large models (the apex, sampled)**: QA + teacher labels for distillation.

Let free CPUs shield expensive GPUs — the recommender cascade transplanted intact. Stacked with the three standing levers (incremental triggering / model tiering / off-peak scheduling), this team's GPU bill drops an order of magnitude.

The three ledgers (final):

Ledger	Contents	Bookkeeper
T4 generation GPU	2.8M turn/s $\times$ ~1 GPU-sec $\rightarrow$ million-GPU scale; the system-wide bulk	The inference team's books + per-user quota set by product (we only report consumption — the meter, not the tollbooth)
T5/T6 refinement	Near-zero after CPU-ization: CPU batch + sampled GPU apex	This team's quota
CPU/RAM	4000-blade hot pool, shards, batch jobs	This team's first-class resource

The leverage thesis (interview line): "We cannot save T4's money, but we decide whether every T4 dollar is well spent — context assembly quality (the right memories, the fewest tokens) directly determines the answer quality and prefill length of that million-GPU fleet. We are the lever on the system's largest bill, not its payer."

9. The Resource Map and Budget Negotiation

9.1 Hot-tier four-tenant ledger (final)

Tenant	Steady state	Quota/buffer
Session state	~20TB	50TB
Facts + embeddings (with 2-3 $\times$ version coexistence)	~100TB	200-300TB

Tenant	Steady state	Quota/buffer
In-flight context	~10TB	50TB (cap basis)
Subtotal (T0–T4 state-class)	~130TB	~400TB (hard floor)
Retrieval hot indexes (conversation inverted/vector hot portion, ~1% LRU)	—	remaining ~1.6PB — the majority tenant

Conclusion: the whole T0–T4 family, buffers maxed, is 20% of the pool; **2PB's substance is always the retrieval index** — the budget's real battlefield is index admission policy (which users/chunks earn RAM).

9.2 2PB's identity: an application, not a grant

2PB is a self-set anchor (the classic 1%-hot working-set point). Negotiation frame:

- **Bottom-up:** hard floor ~400TB (state-class tenants; the retreat line = all indexes sink to SSD, P99 rises a few ms — the product survives, the experience downgrades); the elastic segment is the retrieval index (each added PB buys X% RAM hit rate, X from trace simulation);
- **Top-down:** company RAM pool ~hundreds of PB (a million machines $\times$ 256-512GB); reference points — recommendation (sparse embedding tables) tens of PB the largest tenant, search/ads ~10PB each; the agent line's sane landing zone **1-5PB** (~1% of the pool, commensurate with a 1B-DAU strategic product);
- **The cost-structure inversion (the answer one grade up):** "RAM is not this business line's scarce resource — GPU is; the hot tier at 1-5PB is single-digit percent of the inference fleet's cost, so the design principle is **trade cheap RAM for expensive GPU** (cache more, recompute less, reuse context) — the direction of that trade is itself the budget argument."

This team's resource identity card: RAM/SSD/HDD are first-class (planned, requested, optimized in-house); GPU/VRAM is rented (inference-side interface + quota; this team is a caller; how VRAM is carved and KV caches managed stays across the border). Survival strategy: **hedge the rented resource's scarcity (GPU) with the first-class resources' abundance (RAM largesse, idle CPU cores).**

9.3 Global GPU reference frame (mid-2026)

~16-20M H100-equivalents deployed/sold globally; at ~100GB/card, **global HBM $\approx$ 1.5-2 EB**. By house: Google ~5M H100e (~25% of global compute) $\rightarrow$ ~400-500PB HBM; OpenAI ~1.7M (mostly rented via Microsoft/Oracle/CoreWeave) $\rightarrow$ ~150PB; Meta ~1M-class $\rightarrow$ ~100PB; xAI Colossus 555K cards (~\$18B, 2GW site) $\rightarrow$ ~50-80PB, at ~11% utilization vs. Meta/Google's 43-46% (buying and saturating are different skills). **A tier-one company's VRAM pool alone (100-500PB) dwarfs this team's 2PB RAM application by two orders — "GPU scarcity" is about compute, procurement, and power, never its storage; this team's ask is a rounding error on the company map — low negotiation difficulty is itself an architectural virtue.**

10. Methodology Deposits (the discipline record)

1. **Anchoring discipline:** one master anchor; every derived number collides with it; changing it scales everything; never mix. Reporting posture: range + sensitive variable ("TB–10TB, depending on the activity coefficient — needs data") beats false precision.
2. **Denominator check:** for any "rate," first ask what the denominator is (per access vs. per segment — an order of magnitude apart).
3. **Blunt beats clever (three in a row):** TTL/LRU over session-aware eviction; ADD-only + read-time adjudication over write-time reconcile machines; trace replay over queueing closed forms. Clever schemes tend to couple two paths that were independently decoupled.
4. **Mechanism vs. policy:** this team owns pipelines/schemas/scheduling (how); the algorithm team owns judgment standards (what). Saying the boundary aloud is itself a scoring move.
5. **Analysis proves structure; simulation produces numbers:** Little's Law/conjugacy/80-20 answer magnitude and causal direction; trace-driven simulation answers exact figures + quantiles + sweeps, hung as a periodic regression.
6. **Industry alignment first:** after every self-derivation, check what practice actually does (mem0, Discord, Redis references, GLiNER, adaptive RAG); annotate each deviation — all four "template kills" happened right after alignment moves.
7. **Three fates of a decision:** eliminated (cost < judging cost $\rightarrow$ always do), amortized (ride a call that happens anyway $\rightarrow$ tool call), retained (build a controller only at genuine scarcity).
8. **Little's Law, thrice:** concurrent heads (DAU $\times$ hours/24), resident bytes (inflow $\times$ residency), in-flight requests (QPS $\times$ response) — flow times residence equals stock, one identity in three dimensions.

9. **The same idea grows once per adjacent layer:** delta read (data) and prefix caching (inference); inverted index (rebuildable yet stored) and embeddings (materialized index). Naming the isomorphism halves the explanation.
10. **Depth-to-purpose (the prior session's master principle):** estimation scale, optimization trade, whole-question depth, cross-topic cognitive allocation — four calibrations of one skill; "this question closes at interview depth: for every subsystem I know what lies below, can follow when asked, and don't dig unprompted" is itself a sayable interview line.

11. Interview Quick Card

- **Opening spine:** dual-write (online KV + warehouse) → offline refinement → backfill to online retrieval; critical path reads only.
- **No-KV in one line:** recomputable + $10^5 \times$ dearer + model-bound + single-session reuse; store raw text (ground truth) + context snapshot log (unreplayable; the training-data staple).
- **Retrieval in one line:** v0 lexical + rewriting (same-person vocabulary + BEIR + zero training); dense is the v1 after signal building; the five-tier signal cascade + reformulation-as-reward; per-user small corpora take brute-force dot products, no ANN.
- **T0 in one line:** ordered KV/LSM, (userid, sessionid, bucket), 2h sliding TTL + jitter + volatile-lru + 50TB quota; 1.5% misses at segment heads locked by circadian rhythm; capacity argued threefold; parameters from trace replay.
- **T1 in one line:** hot facts always injected (decision eliminated); cold retrieval as tool calls (decision amortized); the pre-router only prefetches.
- **T5/T6 in one line:** four async per-turn writes; T6 = refinement + compaction, two legs (LSM analogy); new-old knowledge in three cases = merge / mark-and-read-adjudicate / decay-and-compress; costs = a CPU-encoder funnel with a GPU apex.
- **Resources in one line:** RAM magnate, GPU tenant, T4 lever — "we decide whether every dollar of the biggest bill is well spent."
- **Debt list (next sessions):** T6 mechanism deep-dives (contradiction materialization / decay functions / pyramid triggers), multimodal fusion, routing, mem0/memOS source skims, the unanswered event-tracking-pipeline mock, the owed reconciliation mock.

12. Seven Invariants Running Through Everything (the ascension layer)

The system looks like a hundred decisions; strip the nouns and only seven ideas keep instantiating. Recognizing the isomorphism is the step from "memorized conclusions" to "seeing through a class of systems" — and the moment in an interview when you connect two apparently unrelated sub-problems with "these are the same thing" is the moment the assessment jumps from senior to staff.

12.1 Invariant One: $L = \lambda W$ (flow integrates into stock)

One identity, three dimensions, four appearances:

Appearance	λ (flow)	W (residence)	L (stock)
Concurrent users	11.6K sessions/s	4h	167M concurrent sessions
Hot-pool bytes	5.5 GB/s (480TB/day)	~4.5h (session lifetime + TTL tail)	~90TB raw → 30-45TB compressed
In-flight requests	2.8M prompts/s	~10s response (60s cap)	8TB typical / 50TB cap
Inference KV VRAM	turn arrival rate	prefill+decode duration	concurrency $\times$ context length $\times$ per-token KV (another team's books, same law)

Its deep value is **dispelling one class of hallucination**: "a day's 480TB must all sit in hot storage" mistakes the daily integral of λ for L. Whenever a large number appears, first ask: is this flow through the system, or stock within it? Stock always = flow $\times$ residence. A pipe's girth is set by flow rate $\times$ transit time — never by how much water passes in a day.

12.2 Invariant Two: the recompute trichotomy (data's identity is decided by recompute economics)

Recomputable and cheap → don't store; recomputable but expensive with high reuse / structural need for precomputation → store as cache/index; non-recomputable → must store as log.

One criterion table settles every "store or not" dispute:

Object	Recomputable?	Recompute cost	Reuse pattern	Verdict	Identity
Raw text	No (world input)	—	Source of everything	Must store	ground truth
Context snapshot	No (function of time-varying state)	Unreplayable	Trains the memory system	Must store	log
Embedding	Yes	Cheap (~1PB)	One vector serves all future queries + index must precede query	Store	materialized index
Inverted index	Yes	Cheap	Same	Store	materialized index
KV cache	Yes	Prefill cheap (compute-bound)	Single-session + model-bound	Never warehoused	minute-scale serving cache
QKV as training data	Yes	Marginal information = 0	Cannot enter backprop	Don't store	hallucination
Model	Yes (training artifact)	—	A few registry copies	Store a pointer	artifact

The deep sentence: **data's worth is not "does it contain information" but "is the information already elsewhere, and what is the cost structure of getting it back."** Activations are the model's reaction to text, not the world's message to the model; raw text is the sufficient statistic.

12.3 Invariant Three: blunt beats clever (the victory of decoupling)

Four duels, four blunt wins, one common cause — **the clever scheme secretly couples two independently decoupled paths:**

Clever scheme	Blunt scheme	What the clever one secretly coupled
Session-aware eviction ("evict after refinement")	2h sliding TTL + volatile-lru	RAM eviction to the refinement pipeline — which consumes the warehouse, never the RAM copy
Write-time reconcile machine (UPDATE/DELETE)	ADD-only + temporal metadata + read-time adjudication	Write latency to semantic judgment — which can defer to read; writes must be fast
Composite key (taskid in key)	(userid, sessionid) + bounded read + app filter	Storage layout to a computation not yet performed (classification) — circular dependency
Per-prompt λ controller	Always-inject hot layer + tool calls	Decisions to a standalone component — decisions can be eliminated (cost inversion) or amortized (ride-along)

The blunt parts' shared virtue: each component answers to one simple condition (time, capacity, append, reuse), and components decouple through "reloadable / replayable / rebuildable" — **any layer can be lost and rebuilt from the layer below, therefore any layer may be aggressive.** This is the divide between a design that runs ten years in production and one that merely looks good in review.

12.4 Invariant Four: conjugate pairs (every gain wears a price tag)

No free knobs; each connects a conjugate pair:

- **miss $\leftrightarrow$ memory:** uniform access = zero miss at 110TB universal residency; bursty = 3-4 segment-head misses buying 20TB. The user's sleep is the memory budget; misses are the receipts;
- **compression $\leftrightarrow$ access pattern:** 8:1 batch blocks buy sequential-only; 2-3:1 records buy random access. The 160TB-vs-60TB spread is random access's sticker price;
- **tokens $\leftrightarrow$ index:** agentic search spends tokens to save an index; embeddings spend an index to save tokens — the Claude Code vs. Cursor split is this pair solved in two QPS regimes;
- **write latency $\leftrightarrow$ consistency debt:** T5's append escapes latency by borrowing dedup/contradiction/thrift as debt; T6 compaction repays — LSM's soul;
- **network tax $\leftrightarrow$ elasticity:** compute-follows-data waives the read tax but locks two scaling cadences together; stateless serving pays 12GB/s for independence.

The high-scoring interview posture is not "picking the right side" but **laying the conjugate pair on the table, pointing at where your knife falls, and why**. "TTL 2h→8h buys one point of hit rate for 4× memory — not worth it" carries the whole framework in one sentence.

12.5 Invariant Five: the funnel (let the free shield the expensive)

The same cascade grows independently in four places:

1. **Retrieval**: lexical (free) → vector dot product (cheap) → LLM rerank (dear) → generation (dearest);
2. **T6 judgment**: hash dedup (free) → embedding similarity + entity match screening (cheap CPU) → small NLI (CPU 50ms) → LLM for hard cases (GPU) → large-model spot checks (GPU apex);
3. **Signal production**: execution feedback (free, voluminous — the staple) → sampled utilization labels → gold-set calibration (dearest — the referee only);
4. **Decisions**: always-inject (eliminated) → tool call (amortized) → pre-router prefetch (optional accelerator).

The recommender's multi-stage ranking transplanted intact. **One design rule for funnels: each stage's duty is to ensure the next, dearer resource sees only candidates worth seeing.**

12.6 Invariant Six: the same idea grows once in each adjacent layer

- **delta read (data layer) ↔ prefix caching (inference layer)**: both are "the session's shared prefix already sits near; ship/compute only the increment." One cuts KV→serving from 300 to 12GB/s; the other cuts serving→GPU from 840 to 30. The interface in one sentence: session-affine routing + identifiable increments (seqno);
- **LRU (cache layer) ↔ return-probability estimation (statistics layer)**: LRU is not a "policy" — it is an online nonparametric estimator of the return-gap distribution, which is why the heavy-tailed CDF never needs offline solving;
- **inverted index (term space) ↔ embedding (semantic space)**: both are "rebuildable from text yet universally stored" materialized indexes; hybrid retrieval is rank fusion across the two index spaces;
- **trace replay (capacity) ↔ counterfactual ablation (signals)**: both are "the closed form is hard, so replay another world on historical data."

Naming "this is X's mirror at layer Y" halves the explanation and reveals comprehension of mechanism rather than nouns.

12.7 Invariant Seven: the boundary is the design ("what we don't do" is a conclusion, not a disclaimer)

The most underrated move in the whole exercise is boundary-drawing — yet every boundary reshaped the design itself:

- **TAO/UI carved out** → inference-side reduces to raw text + log; only then does the trichotomy have traction;
- **ANN/global retrieval carved out** → per-user brute force becomes the terminal solution; 5000-dim over-designs lose their soil;
- **Generation-model choice and the global GPU pool carved out** → the λ controller dissolves; T1 collapses to two layers;
- **Mechanism vs. policy split** → leg two stays in-bounds while judgment models go next door — the team's *raison d'être* (neither pure storage nor algorithms) is defined exactly by this seam;
- **RAM first-class vs. GPU rented** → "trade cheap RAM for expensive GPU" turns from slogan into the budget argument itself.

The final watershed of system-design interviews: **juniors treat boundaries as "no time to cover"; seniors as "known but not expanded"; staff say the boundary aloud as a design conclusion** — "this question closes at interview depth: for every subsystem I know what lies below, can follow when asked, and don't dig unprompted."

13. The Number Derivation Tree (every figure's pedigree from anchor to leaf)

Any number spoken in an interview should trace back to the anchor on demand. Conversely, change the anchor and the whole tree refreshes by the leaf multipliers — the visualization of "re-anchoring is a global event."

```
Anchor: 1B DAU × 240 msgs/day × 2KB/msg
|
|— Flow branch
|   |— 480 TB/day raw (×5:1 → warehouse ~96TB/day, 60-100TB band)
|   |— 175 PB/yr raw → 35 PB/yr compressed → cold EC 1.4× ≈ 49 PB
```

- | — 2.8M msg/s mean ($\div 86400$) $\rightarrow$ ~8M/s peak ($\times 3$)
 - | — Read bandwidth = $2.8M \times \text{context size}$
 - | — $\times 100KB$ (full session-state read) = 300 GB/s $\rightarrow$ 12 GB/s after delta read
 - | — $\times 300KB$ (assembled context $\rightarrow$ GPU) = 840 GB/s $\rightarrow$ ~30 GB/s after prefix cache
- | — Concurrency branch (Little's Law)
 - | — Concurrent sessions = $1B \times 4h/24h = 167M$ (sensitive var: daily hours; 30min tier $\rightarrow$ 20M)
 - | — Session arrival = $1B/86400 \approx 11.6K/s$ (= the miss-rate numerator)
 - | — In-flight = $2.8M \times \text{response time}$ (10s $\rightarrow$ 28M; 60s cap $\rightarrow$ 168M)
- | — Stock branch (= flow $\times$ residence)
 - | — Session state = $240 \times 2KB = 480KB$ raw $\rightarrow$ ~110KB compressed (with centroids/metadata)
 - | — Hot-pool steady state = $5.5GB/s \times \sim 4.5h \approx 90TB$ raw $\rightarrow$ 30-45TB (quota 50TB)
 - | — Fact layer = $300KB \times 1B = 300TB$ total; hot = $167M \times 300KB \approx 50TB$ (quota 200-300TB w/ versions)
 - | — In-flight context = in-flight $\times$ 300KB (8TB typical / 50TB cap)
 - | — Pool total: state-class ~130TB steady / 400TB floor; retrieval indexes take the remaining ~1.6PB
- | — Miss branch
 - | — miss/s $\approx$ arrival of segments with gap>2h $\approx 40K/s$ (3.5 seg/user/day, circadian-locked)
 - | — miss rate = $3.5/240 \approx 1.5\%$ (0.4%-2.5% band; extreme ceiling 8%)
 - | — Reload flow = $40K \times 100KB = 4 GB/s$ (durable-tier noise)
- | — Machine branch
 - | — Hot pool = $2PB \div 500GB$ usable = 4000 blades (session state parasitizes 13GB = 2.6% each)
 - | — KV by QPS = $2.8M \div 100K$ ops = 28 mean / 80 peak (bandwidth-driven ~100; delta read relaxes)
 - | — Inference = $2.8M \times \sim 1 GPU\text{-s/turn}$ $\rightarrow$ million-GPU scale (another team's books; reference frame)
- | — Refinement branch
 - | — T5 extraction = 2.8M/s (per-turn — this team's top line $\rightarrow$ near-zero GPU after encoder-ization)
 - | — T6 batch $\approx 10^9$ users $\times$ a few judgments/day $\approx 10\text{-}20K/s$ (off-peak troughs)
 - | — Per-user T6 round $\approx$ seconds (dot products μs + NLI ~100ms + triggered summaries seconds)

Three usage disciplines: state the anchor before any number ("on the 240/day anchor"); report ranges with sensitive variables; when challenged, walk back up the tree instead of re-guessing.

14. The Fifteen-Minute Spoken Skeleton (live-interview edition)

The whole manuscript compressed into interview-room speech — each block gives the density and order of what is actually said (parentheses = accompanying action). Reality drills down wherever the interviewer interrupts; the trunk stays fixed.

[0-2 min] Restatement and boundary declaration. "I understand this as the agent's memory subsystem: upward it exposes ingest and retrieve; downward it rents storage engines and GPU inference; its neighbors are orchestration, inference serving, and model training — three areas I'll touch at the interface only. Anchoring scale: 1B users, 240 messages each daily, 2KB each — 480TB/day, 2.8M QPS; every number that follows derives from this." (Anchor in the top-left of the whiteboard; the sandwich boundary top-right.)

[2-5 min] Data plane and "what to store." "First, what to store. The criterion is recompute economics. Non-recomputables must be stored — raw text, and the inference-time context snapshot, which is a function of time-varying state, unreplayable afterward, and also the core training data for retrieval ranking — the analogue of ads feature logging. Recomputables that structurally require precomputation are stored as indexes — embeddings and inverted lists. Recomputables that are expensive and single-session are not stored — KV cache is 10^5 times the text and voided by model upgrades; it deserves only minutes of engine-side caching. Tiering: hot RAM for active state and hot indexes; warm SSD three months at 5:1, about 9PB; cold HDD full history with EC, about 50PB a year. Compression is a tier property — big blocks buy ratio, small records buy random access."

[5-8 min] Read path and retrieval. "A prompt's journey: read session state (ordered KV, userid+sessionid shards, 2-hour sliding TTL with LRU — tens of TB steady, 98%+ hits, misses pinned to segment heads by circadian rhythm at ~1.5%); cheap memory — profile and high-confidence facts — injects unconditionally every turn, no decision, since judging costs more than injecting; expensive retrieval is exposed as tools the generation model calls itself, the decision riding a call that happens anyway. Retrieval proper: a per-user corpus is $\sim 10^5$ vectors — brute-force dot product is terminal, no ANN. v0 is lexical plus LLM query rewriting — personal memory has query and corpus in one person's vocabulary, and BEIR showed BM25 beats dense without in-domain data. Dense is the v1 after utilization signal accrues — this scenario has no native click: no SERP, per-user corpus, inferred labels — so signal building is itself the hardest sub-problem."

[8-11 min] The write side — this team's heart. "All writes async; the critical path only reads. T5 per turn, four items: append state, dual-write the warehouse, land the context snapshot, extract facts — extraction on CPU encoders (the GLiNER family matches LLMs and runs on our blades' idle cores), GPU only for hard cases. T6 is compaction: an ADD-only library rots in six months — duplicates, contradictions, staleness are the three new-old relations, handled by merging, temporal metadata with read-time adjudication, and decay with quota compression. The LSM analogy: T5 is memtable append escaping write latency by borrowing consistency debt; T6 repays. On forgetting: TTL governs caches; supersession and decay govern memory — an allergy must not expire after ten quiet years."

[11-13 min] Cost structure and the resource map. "Three ledgers: the system's bulk is generation — million-GPU scale on the inference team's books; we only report consumption. Our own refinement quota is near zero after CPU-ization. RAM is our first-class resource — the hot tier asks for 2PB: a 400TB hard floor of state, the elastic segment being retrieval indexes, the exact figure negotiated on a trace-simulated hit-rate-versus-cost curve. Design philosophy in one line: trade cheap RAM for expensive GPU — every cache hit is one less inference call; and we decide whether every dollar of the biggest bill is well spent — context quality determines what that million-GPU fleet produces."

[13-15 min] Close and planted hooks. "Every component here has a zero-training v0, and the logs are laid from day one for v1 — the system runs first, and the signal grows out of it. What I haven't expanded: contradiction materialization, the summary pyramid's triggers, multimodal fusion, the GDPR cascade — each goes one level deeper on request." (Every hook genuinely has a level prepared.)

Three all-purpose moves when interrupted: state the anchor before the number, then range and sensitive variable; when challenged, collide with an invariant before revising; when pulled out of bounds, give the interface in one sentence and ask "shall we expand that side?"

Appendix A: Derivation Records of the Key Conclusions

The main text gives conclusions in static form; this appendix restores how each was forced into being — initial hypothesis, magnitude test, counterexample, revision, convergence. The derivation paths themselves are the reusable asset: reskin the problem and the same paths carry.

A.1 Deriving "what to store": from "store everything" to the trichotomy

Initial hypothesis: store everything inference-related — raw conversations, raw QKV, the corresponding model — for at least a month, as future training data.

The first magnitude test eliminates QKV immediately. Unit cost first: Q is never cached (computed fresh per token; attention caches only K/V); at 70B scale, per-token KV = $2 \times 80 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ dims} \times 2 \text{ bytes} \approx 320\text{KB}$, versus ~ 4 bytes of raw text — $10^5 \times$. Against the anchor: $1.2 \times 10^{13} \text{ tokens/day} \times 320\text{KB} \approx 4 \text{ EB/day}$, 120 EB per month — exceeding a tier-one company's whole warehouse. Last-layer-only still costs 32KB/token $\rightarrow$ 380 PB/day $\approx$ two years of compressed text daily. The storage ledger kills it outright.

But the magnitude ledger only answers "affordable," not "worthwhile" — so the follow-up: were it affordable, is the last layer's QKV good training data? This pushes the derivation into information theory: activations are a **deterministic function** $f(\text{raw text, model weights})$, precisely recomputable, hence zero marginal information relative to raw text — not data but cached computation. Training-data value lies in what the model lacks (what the user said, which answer was accepted), all of which lives in the text. Nor does storing help training mechanically: backprop rebuilds all lower activations in the forward pass; a stored last layer has no slot. **The intuition's legitimate kernel is rehoused in two proper outlets:** soft labels $\rightarrow$ top-k logits ($\sim 100\text{B}/\text{token}$, genuinely informative to students lacking teacher weights, sampled subsets only); semantics $\rightarrow$ pooled embeddings (1–4KB/chunk).

A second, reversed probe: "surely inference-time features must be stored — while embeddings needn't be, the model can regenerate them." The mirrored pair completes the trichotomy:

Recomputable and cheap $\rightarrow$ don't store; recomputable but expensive with structural precomputation needs $\rightarrow$ store as cache/index; non-recomputable $\rightarrow$ must store as log.

The context snapshot falls in category three (a function of time-varying system state — which memories were recalled that instant, the profile's snapshot, model version — unreplayable; the exact pattern of ads feature logging). Embeddings fall in category two: marginal information genuinely zero (that half of the probe stands), but the reason to store was never information — ANN structurally demands the whole library's vectors exist before any query, plus one-vector-serves-all-future-queries reuse at $\sim 1\text{PB}$ cost. Identity: **materialized**

index, kin to the inverted list. KV is category one's cautionary tale: recomputable + $10^5 \times$ + single-session + model-bound — a minutes-scale engine cache, never warehoused.

Convergence: long-term inference-side storage = raw text (ground truth) + context snapshot log (training-data staple); embeddings assigned to the retrieval block (materialized index); QKV never; models as registry pointers. One criterion, four objects, four verdicts.

A.2 Deriving retrieval signals: from "no clicks, can't train" to the production line

Initial predicament: the standard dense recipe (two-tower + click logs) lacks its raw material here.

Round one (pseudo-query + LLM-as-judge) fails inspection. Back-generating queries from chunks carries **structural** noise: generated queries are too tidy; real ones are fragmentary, anaphoric, context-dependent. The judge scores (q, chunk) relevance but never q's representativeness. Cold-start pseudo-labels keep v0 from blindness; they cannot underwrite quality claims.

The alternative — "recall wide + LLM summarize-and-select" — dodges training but relocates the bill. Cost test: precision work moves from the millisecond/micro-dollar retrieval layer to the hundred-ms/milli-dollar LLM layer — $10^3 \times$ unit cost; latency test: two serial LLM hops. Yet the scheme has one scenario-specific pillar: **query and corpus share one person's word distribution** — recall-side lexical strength far beyond general search.

The synthesis is temporal division, not either-or: wide-recall + LLM selection ships as v0 (zero training, one week), and the moment it runs, the cost problem becomes the data source — "which chunks the LLM used or discarded" is utilization labeling under the true query distribution, one grade cleaner than pseudo-queries. Dense is the v1 trained on those logs; its *raison d'être* is not quality but narrowing the wide recall to reclaim the LLM bill. BEIR's history endorses the ordering.

The full signal derivation yields the five-tier cascade (main text § 3.3), whose pivotal tier was derived independently: **the next prompt within a session scores the previous query** — restating = negative, deepening = positive, switching = unlabeled. Isomorphic to twenty years of query-reformulation-as-implicit-feedback, with two traps nailed on the spot: attribution confounding (the culprit may be the query, absent knowledge, or generation misuse — condition on execution-feedback tiers to disambiguate); distribution contamination (the next prompt reacts to the observed failure — **reward, never a seq2seq label**).

The per-user-corpus objection tested against a counterexample: "Google search is also per-user personalized" — dissected, it is no counterexample: web search is a shared corpus + per-user features; doc-level signal still pools across a billion users. The true kin is Gmail/Drive personal search, whose literature completes the derivation: doc-level clicks cannot pool → parameterize signal into **attribute space** (term match, recency, affinity) transferring across users. Corollary here: any two-tower's doc side carries no memory-ID parameters — the forced constraint aids transfer and retroactively fortifies lexical v0. The incumbent advantage sharpens likewise: Gmail owns real SERP clicks on per-user corpora, but compliance walls confine the asset to models and methodology; for players without the inheritance, signal building is the moat's construction site — the team's reason to exist.

Unifying query generation into one model: (question, history) → query for both halves. Training ladder: prompt → rejection sampling → RL on verifiable retrieval reward (Search-R1: the query distribution is the policy's own exploration; the pseudo-query gap is bypassed). Unsupervised LM on history contributes exactly the **representation** (vocabulary, style, query-likelihood prior) — but the LM objective ≠ query utility: dialogue never contains search strings, and good queries need discriminative terms while high-probability terms are the least discriminative (why TF-IDF exists). **Representation is free; policy is paid — in utilization logs.** On "replacing embedding retrieval": the Claude Code vs. Cursor split marks the frontier — tokens-for-index vs. index-for-tokens; at 1B users with per-turn retrieval on the critical path, query gen fronts the retrieval layer rather than replacing it. The ledger is QPS.

A.3 Deriving T0: how one session table was reduced to blunt parts

Schema. The first inclination — composite keys with taskid — dies of a circular dependency: taskid is not yet computed at query time (classification depends on the context being read). Keys converge to (userid, sessionid[, bucket]) time-descending, **bounded reads + app-layer task filtering**; the bucket borrows Discord's guard against runaway partitions. Clustering itself ships to the algorithm team; this team signs the systems contract: centroid dot products (single-digit K, microseconds), sticky-to-thread for low-confidence elliptical prompts (they occur precisely when the topic has not switched — the continuity prior comes free), $O(1)$ cluster creation.

Capacity survived three calibrations. First: a "2M concurrent, 10M peak" side-estimate produced 10TB — collision with the master anchor failed (matching 2.8M QPS would need 1.4 msgs/user/sec — absurd); the identity ruled 167M. Second: a unit drift (0.5KB/msg supplanting the 2KB anchor) was hauled back; session state fixed at ~110KB. Third: flow confused with stock — "how can a 480TB day fit in 50TB?" Resolved by Little's Law in a new dimension: stock = $5.5\text{GB/s} \times \sim 16000\text{s} \approx 90\text{TB}$ raw → 30-45TB compressed, locking with the head-count reckoning. The hot pool is a pass-through buffer, not a distillery; "all day simultaneously in RAM" never occurs; 160TB is the counterfactual bound whose premium prices random access.

Eviction was a complete clever-vs-blunt campaign. The clever original: evict on session end + refinement completion. First crack: usage is fragmented (4h/day in morning/noon/evening shards) — per-segment eviction misses evening returns; patched with reloads + LRU natural residency. Second crack, fatal: industry (Discord/Messenger/AWS references) writes **no session-aware eviction at all** — durable LSM primary, generic TTL cache, LRU eviction; the "after refinement" clause coupled two paths that were never coupled (refinement consumes the warehouse). **The originally-opposed "2h TTL cap" is fully vindicated:** the value sits mid-mainstream; the fixed-vs-sliding dispute was a strawman — chat workloads reset expiry on every write, making the two identical.

Miss rate exposed a denominator trap. An external AI's hit-rate table (continuous 90% / four-segment 25% / extreme ~0%) ranked qualitatively right, quantitatively wrong — per segment instead of per access, discarding the in-segment hundreds. Re-cut: 98%+ everywhere; "~0%" physically impossible under the anchor. True structure: sliding-TTL misses live only at segment heads with gaps > 2h, whose count circadian rhythm locks at 3-4/day → ~1.5%, insensitive to fragmentation.

The uniformization thought experiment sealed it: uniform arrivals (6-minute gaps « 2h) never expire — zero miss — but universal residency = 110TB blows the quota. **Miss and memory are conjugate; the user's sleep is the memory budget; segment-head misses are the savings receipts; TTL is where the knife falls.** The capacity case closed threefold: means (Little's Law), distribution (conjugacy), ordering (80/20 — LRU spends memory on traffic magnates; flow-weighted hits > 99%).

Exact vs. engineering: the linear-expectation formula for 30-day data is an undeclared approximation — residency is a functional of the gap distribution (the clump process under sliding TTL: the same 4h/day uniform vs. three-block differs 4×), heavy-tail analysis is hard. Engineering declines to solve: **trace-driven simulation** replays the logs through TTL+LRU; steady state, misses, quantiles, τ sweeps all read off; hung as periodic regression. **Analysis proves structure; simulation produces numbers** — what cannot be proven directly gets simulated.

A.4 Where data lives, where compute lives: the complete NIC derivation

The line starts from a naive question — "reading session state is a CPU reading memory; what has the NIC to do with it?" — and in five steps forces the architecture down to physics and back up to organization.

Step one, supply the missing subject. "The CPU reads local memory" lacks whose CPU: by default the reading CPU sits on a serving node while the data sits on a KV node — every 100KB read travels KV RAM → CPU serialization → NIC → switch → serving NIC. The 300GB/s aggregate ($2.8M \times 100KB$; reads out-byte writes 50:1 — "read-heavy means bytes") lands on 80 KV nodes' egress at 3.5GB/s each — 25GbE saturated.

Step two, split the merged decisions. "Prompts carry userid — assign machines directly": routing by userid (one config line; always do it) $\neq$ co-locating serving with shards (the actual tax switch). Routing without co-location still crosses two NICs.

Step three, strip to physics. One criterion: same machine → memory bus (~100GB/s, 100ns); across machines → two NICs (~3GB/s, 100 μ s), 30× slower. This is the physical divide between compute-follows-data (WhatsApp/Orleans lineage) and stateless-serving-plus-remote-KV.

Step four, the cost ledger. Co-location's four bills: heterogeneous scaling chained; releases roll stateful nodes; hotspots un-drainable; merged failure domains — plus the fatal one, re-sharding cadence lock (long-stable shards vs. minute-level serving elasticity). The stateless side pays 300GB/s, cut to 12 by delta read (a serving-side session cache + cursor increments; a cache, not a replica — T3 holds that memory anyway; loss degrades to a full read).

Step five, scale flipped the lean. At the 4000-blade frame ($2PB \div 500GB$), compute-follows-data strengthens a grade: session state parasitizes 13GB/blade; a 1TB machine splits 250/500/250 healthily; the per-user family co-locates so T0-T3 run fully local; virtual shards drain hotspots. The "blade→GPU 840GB/s is unavoidable" objection dissolves by boundary-cutting — that is the inference interface's ledger (and tens of MB/s per node across the fleet; prefix caching cuts it to 30GB/s — delta read's mirror), leaving "the session-state read's NIC drops to zero" intact.

Convergence: two legitimate hills; the choice is organizational. Interviews default to stateless + delta read; compute-follows-data stays pocketed for extreme latency budgets. **Routing affinity yes; deployment affinity depends. CPU compute follows data; GPU compute follows the batch.**

A.5 Deriving T1: the birth and dissolution of a controller

It began with a handsome structural recognition: "each prompt decides whether to search, which machinery tier, how much context — this smells like pacing." The mapping holds completely: GPU quota = daily budget, action bundles = bids, v/c greedy + dual λ = shadow price, PI controller on the burn curve.

Three brakes dismantled it. First: against the JD — two decision variables out of bounds (generation-model choice; the global GPU pool); the controller shrinks into the pipeline's own quota. Second: "the problem got easier" — no, the difficulty relocated into estimating v (predicting in microseconds, at zero GPU, an outcome that costs hundreds of milliseconds to observe). Third, fatal: "stop templating — check industry." Industry builds no such component: cheap memory **injects unconditionally** (judging costs more than injecting — decision eliminated); expensive retrieval is **a tool the generation LLM calls** (the judgment rides a call that happens anyway — decision amortized). The "2.8M/s of judgments" problem was a pseudo-problem.

The pre-router survives with a demoted identity — prefetch trigger, not decider (Bing Chat's precedent); wrong guesses cost a wasted search or an agent retry. **The router manages speed; the tool call manages correctness.** λ survives as one bone: tiering by cost.

The meta-lesson: the templating temptation peaks exactly where structures are isomorphic — the pacing math truly is isomorphic, but industry's two blunter moves (eliminate, amortize) leave the controller with no reason to exist. **Structural isomorphism $\neq$ component isomorphism.**

A.6 Deriving T5/T6: from "extract facts" to "maintain a rotting library"

T6's first definition ("user-level fact and embedding extraction") covered only the outputs. Asking "if per-turn T5 does all it can, what remains?" forced out the substance: everything requiring **global vision or batch economics** — fixed by one analogy: **T5/T6 = LSM memtable append / compaction**. T5's append escapes write latency (ADD-only, message-pair myopia), borrowing consistency as debt; T6 repays. The default of not repaying: silent precision collapse.

The "leg two is business logic" boundary dispute, adjudicated by JD verbatim: compression/forgetting/conflict-resolution/lifecycle are leg two's four items by name — carve it out and the team degenerates into pure storage, the neighbor's turf. Yet the instinct's valid half was re-cut as **mechanism vs. policy**: pipelines/schemas/scheduling/replay in-bounds; contradiction definitions/decay coefficients/importance next door — recommender infra builds the feature store, never the ranking objective.

The unified skeleton surfaced through definitions: three new-old relations — duplicate (dot-product merge = entity resolution), exclusive (ADD-only + temporal metadata + marks, read-time adjudication, hot pairs materialized), stale (importance $\times$ confidence $\times$ decay; quota triggers merge-to-summary — precision down, never zeroed). The forgetting ontology set: **TTL governs caches; supersession + decay govern memory.**

The cost model took three corrections. First: "knowledge-graph building — light compute, ten seconds" — right per user, wrong $\times 10^9$ (the standing GPU-bulk ledger returns); and the "60 seconds of system time" prepaid nothing — **RAM-seconds (occupancy) and GPU-seconds (work) are separate books**, non-overlapping in time and kind. Second, by frequency: the team's top line is actually T5 per-turn extraction ($200 \times$ T6's cadence). Third, "can CPUs do this?" checked against industry: GLiNER-family encoders, distilled BERTs, TinyBERT NLI — the funnel forms (CPU 80-90% on the blades' idle troughs $\rightarrow$ GPU small for hard cases $\rightarrow$ GPU large sampled), cutting the team's GPU bill an order of magnitude. **Free CPUs shield expensive GPUs.**

The three ledgers finalized under serial cross-examination: T4 is of course the system-wide bulk — on the inference team's books, per-user quotas set by product, this team a meter not a tollbooth; T5/T6 near-zero after encoder-ization; CPU/RAM first-class. The leverage thesis closes it: "we decide whether every dollar of the biggest bill is well spent."

A.7 Deriving the budget: 2PB from thin air to a negotiation frame

Three successive questions performed the upgrade. "Is 2PB the company's or this line's?" — provenance: a self-set anchor (the 1% working-set point); identity: **an application, not a grant**. "Search/ads/feed/games all want hot memory — what's this line's ceiling?" — both directions: bottom-up hard floor ~ 400 TB (state tenants; retreat = indexes to SSD, P99 up a few ms); top-down company pool $\sim$ hundreds of PB, recommendation the largest tenant at tens of PB, the agent line's sane zone **1-5PB**. The last question fixed the identity card: "this team is chiefly RAM; GPU/VRAM is another team's interface plus quota" — first-class resources planned in-house; the rented one worked only from the calling side. The cost-structure inversion is the grade-up answer: this line's scarcity is GPU, not RAM (tier-one VRAM pools of 100-500PB dwarf the 2PB ask by two orders); **"trade cheap RAM for expensive GPU" is itself the budget argument.**

A.8 The derivation-move inventory

1. **Magnitude test first:** any store/do decision starts at unit cost $\times$ scale — schemes whose ledgers fail need no value debate;
2. **Collide with invariants:** conflicting estimates are adjudicated by model-independent identities (total message flow ruled the concurrency dispute); new numbers collide with the master anchor before acceptance;
3. **Check the denominator:** every "rate" first declares per-what;
4. **Flow vs. stock:** large numbers first classified; Little's Law works in three dimensions (heads, bytes, in-flight);
5. **Split orthogonal decisions:** unbundle merged choices (routing vs. deployment affinity; quota existence vs. bookkeeper);

6. **The recompute trichotomy:** data's identity from recompute cost $\times$ reuse pattern, never from "contains information";
7. **Clever schemes face industry first:** four clever designs (session-aware eviction, composite keys, the λ controller, extra warm-tier splits) all fell to "what does practice do"; blunt parts won four straight — cleverness tends to couple what was decoupled;
8. **Separate an intuition's kernel from its vehicle:** "the last layer holds information" (true) $\neq$ store QKV (false); when a counterexample kills the vehicle, check whether the kernel deserves a proper outlet;
9. **Put exact and engineering solutions on the table together:** admit the approximation (linear expectation vs. the clump functional), then route around via simulation;
10. **Boundaries are part of the derivation:** mechanism vs. policy, in-bounds vs. neighbors, first-class vs. rented — every boundary spoken aloud is a design conclusion.

Appendix B: T6 Mechanism Pre-Study (advance payment on the debt list)

The main text erected T6's skeleton; this appendix pushes each mechanism one level down — to "can sketch the schema and pseudocode when pressed" — while holding the mechanism-vs-policy line (judgment-model quality belongs next door).

B.1 The complete fact schema (the carrier of every mechanism)

```
fact {
  fact_id, user_id
  text      # plaintext substance: "user lives in Jersey City"
  entities[] # normalized entities (Jersey City → LOC:xxx) — keys for dedup & contradiction detection
  t_observed # event time (when the user said it)
  t_ingested # system time (when it landed) — dual timestamps; adjudication uses t_observed
  confidence # extraction confidence (encoder-scored)
  importance # importance prior (health/identity high, small talk low) — policy parameter, algorithm team's
  hit_stats  # last-K retrieval hits/adoptions with timestamps — the basis of decay & eviction
  supersedes[] # pointers to facts this one replaces (the "soft UPDATE" under ADD-only)
  contradicts[] # contradiction marks (symmetric edges)
  source_span # provenance: back-pointer to raw text (session, seqno) — replay & GDPR both hang on it
  emb_version, embedding
}
```

Two key choices: **dual timestamps** (a March event ingested in November adjudicates on event time); **source_span** (every fact traces to its raw text — replay, explainability, and cascade deletion share this one thread).

B.2 Contradiction adjudication: detect — mark — read-adjudicate — materialize

Detection (T6 batch, funneled): candidate pairs = same entity + embedding similarity in the middle band — identical is a duplicate (merge path); unrelated needs no judging; only the middle band can contradict. Screened pairs enter a small NLI model (CPU grade) → entail/neutral/contradict. Keep the canonical counterexample ready: likes coffee / likes dark roast = refinement (entail); works at Meta / laid off = state transition (contradict + temporal order).

Marking (write side, ADD-only): contradictions write only symmetric edges; both facts live.

Read-time adjudication (default path): when retrieval hits a contradiction-linked set, sort by `t_observed` — newest is current state, old ones down-weighted but **never hidden** ("used to work at Meta" remains a valid historical fact — decisive for a "fix my resume" query). Placing adjudication at read costs a small toll per retrieval, buying zero write blocking + upgradable policy (change read code, never migrate data).

Materialization (T6 optimization, optional): when `hit_stats` show a contradiction pair queried frequently, batch-pre-adjudicate once and store the verdict (a `current_state` pointer) — the materialized view of read-time adjudication. The trigger is frequency, not existence: cold pairs adjudicate at read forever.

B.3 Decay and compression: one score, two actions

Score form (coefficients to the algorithm team; the form to this team, because it dictates stored fields):

```
score = importance  $\times$  confidence  $\times$  recency_decay( $\Delta t$ ), recency_decay =  $\exp(-\lambda_{\text{type}} \cdot \Delta t_{\text{since\_last\_confirm}})$ 
```

Two essentials: **λ tiered by fact type** — identity/health $\lambda \approx 0$ (decade-proof), preferences medium, episodic ("interviewing lately") fast; **Δt runs from last confirmation, not first observation** — every retrieval adoption or user restatement resets the clock, so frequently-useful facts live long. Together they realize "allergies never expire; stale interview chatter sinks naturally" — no explicit TTL required.

Two downstream actions: retrieval ordering (score as one rank-fusion signal); quota compression (on hitting the per-user cap, the lowest-score cluster of same-topic shard-facts merges via one LLM call into a summary fact whose supersedes[] points at the members — **precision reduced, never zeroed, recoverable from source_span replay**).

B.4 The summary pyramid: triggered, not calendared

Levels: turn (T5 extracts facts, no summaries) → session summary (fired on session idle-timeout; itself the context input to T5 extraction — the self-consuming loop) → topic/period summaries (fired only by quota compression or retrieval-side "too many shards on one topic"). **Against calendar jobs** (daily/monthly full sweeps): most users most days have no new structure worth refining; scheduled full passes burn GPU on idling. The pyramid's single top-down reason: **token budget** — the same memory in fewer tokens (the VRAM constraint's shadow cast on storage).

B.5 The GDPR cascade: source_span is the thread

forget(user_id, scope) chains: raw text (warehouse + warm) → session state (TTL natural + active purge) → facts (reverse lookup by source_span across everything refined from the deleted text — including summary facts' transitive closure) → embeddings/inverted entries → context snapshot logs containing the material. Two engineering points: **deletion is async batch with tombstones first** (read paths blind immediately; physical purge rides compaction — the LSM shape again); **the transitive closure rides the supersedes/ source_span graph** — the reason B.1 insists every fact carries its pedigree: deletion compliance is a day-1 schema constraint, not a patch.

Appendix C: The Follow-Up Ammunition Depot (one question, one paragraph, grab-and-go)

Q: Why not have one big LLM read the user's whole history every turn — why retrieval at all? Token budget and money. ~1MB/20 days $\approx$ 250K tokens; reading it per turn multiplies prefill cost a hundredfold and overflows the window. Retrieval exists to "carry 99% of the relevant memory in 1% of the tokens." Long-context models soften but never delete the constraint — cost scales with length; relevant-information density does not.

Q: Vector database choice — Milvus/Pinecone/pgvector? This scenario needs none: per-user 10^4 -5 vectors brute-force in milliseconds; vectors live as ordinary fields in the KV, co-located with the shard. Vector databases solve billion-scale global ANN — recommendation/global search's problem. Selection questions start with scale, not product names.

Q: Concurrent writes from multiple devices? Sessions key on (user, session) — two devices are two sessions, no conflict; the fact layer's ADD-only is natively concurrency-friendly (no in-place updates, no write-write conflicts); dedup and contradictions converge in T6 batch — another append-only dividend.

Q: What is lost when a machine dies? By tier: session state is replica-durable — a millisecond window, at most the last message (user resends); facts/indexes replay from the warehouse ground truth; serving-side soft caches degrade to full reads. Design rule: every layer rebuilds from the layer below, so every layer may be aggressive.

Q: Memory poisoning (user or attacker injecting false facts)? In-bounds mechanisms: facts carry confidence + source_span; low confidence never reaches the hot-injection layer; anomaly detection (fact-generation rate, semantic distance from the historical profile) is one more T6 batch lane; arbitration policy belongs to safety/algorithms. Out-of-bounds in one line: prompt-injection defense lives on the generation side, not in storage.

Q: How does multimodal enter? The skeleton stands; two extensions: raw media in object storage, KV holds links (fixed back at T0); behavioral signals/images pass their own encoders into the same fact/index framework — multimodal fusion's infra form is one more encoder lane plus one more vector space in rank fusion; the hard part (alignment quality) is model-side.

Q: How is the system evaluated? Layered: retrieval — recall@k / utilization rate (gold-set calibrated); memory quality — contradiction rate, duplication rate, fact-freshness accuracy (sampled audits); end-to-end — product proxies like "how often users re-introduce themselves" + memory-related complaint rates; cost — per-turn subsystem spend (GPU calls, RAM hit rate). Trace replay for capacity regression; the gold set for quality regression.

Q: Cold-start users have a poor experience? No history means no memory — a definition, not a defect. Engineering: the first N sessions run pure recent-turns + explicit onboarding ("tell me your preferences" pipes straight into the fact store); the library accrues from turn one. The value curve rises with usage — saying this plainly beats faking cleverness.

Q: The essential difference from ChatGPT memory / mem0? Isomorphic in form (fact extraction + injection + retrieval); the difference is the phase change scale forces: 1B users compel per-user co-located sharding, CPU-encoder-ization, off-peak batch scheduling, trace-simulated capacity — mem0 is a single-tenant library; this is a multi-tenant platform. The weight of the JD's phrase "unified platform" lives exactly there.